붙임 3

연구논문/작품 최종보고서

2026 학년도 제 2 학기

제목 : GPU 프로파일링·시뮬레이션 기반 LLM 추론 attention 커널 동적 선택 시스템

이선재 (2021313805)

GitHub : https://github.com/sunjae0224/kernelscope

2026 년 월 일

지도교수: 민 동 문 서명 _____

계획(10)	주제(20)	개념(20)	상세(30)	보고서(20)	총점(100)

* 지도교수가 평가결과 기재

■ 요약

본 작품은 길이가 서로 다른 요청을 함께 처리하는 LLM 서빙 환경에서 attention 커널의 KV 분할 수를 매 디코드 단계마다 동적으로 선택하는 시스템이다. FlashAttention-2 의 기본 휴리스틱은 배치 크기와 최대 KV 길이만으로 병렬성을 판단하므로, 소수의 긴 요청과 다수의 짧은 요청이 섞이면 일부 스레드 블록(CTA)에 작업이 집중되는 부하 불균형이 발생한다. 이 현상을 분석하고 매 단계의 커널 선택에 활용하기에는 기존 GPU 시뮬레이터와 분석 방법에 한계가 있었다. 사이클 수준 시뮬레이터는 커널 하나를 재생하는 데 수십 초에서 십수 분이 걸리고 실험 GPU 인 RTX 4090 을 지원하지 않았으며, 평균 길이 기반 분석은 요청별 길이 차이를 표현하지 못하였다.

본 작품은 이러한 한계를 보강한 분석 환경을 구축하였다. 요청별 KV 길이로부터 CTA 별 작업량, SM 배정과 메모리 대역폭 공유를 계산하는 CTA 이벤트 기반 성능 시뮬레이터를 구현하고 RTX 4090 실측으로 보정하여 실행 시간 예측 오차 중앙값 2.3~12.3%를 얻었다. 계산 경로를 C 언어로 재구현하여 후보 9 개를 평가하는 첫 선택 비용을 842.6 ms 에서 14.3 ms 로 59 배 줄였고, 시뮬레이션 결과를 매 디코드 단계의 커널 선택에 직접 사용할 수 있게 하였다. 또한 Accel-Sim 에 RTX 4090 자원 구성과 sm_89 명령어 재생 호환성을 추가하였고, GPGPU-Sim 에서는 직접 작성한 split-KV 커널로 혼합 길이 배치의 실행 사이클이 분할에 따라 7.23 배 줄어드는 기전을 재현하였다.

보강한 환경으로 RTX 4090 의 6,801 개 커널 조건을 분석한 결과, 균일 길이 배치에서 기본 휴리스틱의 선택 손실 중앙값은 0.72%였으나 혼합 길이 배치에서는 최적 설정보다 최대 12.53 배 느린 커널이 선택되었다. 실측 테이블 기반 선택 정책을 Qwen3-4B 에 적용하여, 32K 토큰 요청 1 개와 512 토큰 요청 31 개가 섞인 조건의 토큰 간 지연(TPOT)을 62.09 ms 에서 34.25 ms 로 1.81 배, 64K 자연어 문맥 조건에서는 94.81 ms 에서 38.71 ms 로 2.45 배 줄였다. 두 조건 모두 생성 토큰이 기본 설정과 같았으며, 균일 길이 조건에서는 기본 설정과 같은 성능을 유지하였다.

커널을 교체하지 않고 분할 수만 선택하여, FlashInfer 커널로 교체한 경우(1.83 배)에 근접한 개선을 얻었다. 공개 Azure 대화 트레이스를 재생한 분석에서는 디코드 단계의 약 9%(실측 표본 추정)~13.5%(시뮬레이터 예측)가 이러한 취약 조건에 해당함을 확인하였다. 주요 수치는 GPU 없이 원본 기록에서 다시 계산하는 948 개 검증 항목으로 확인하였다.

키워드: LLM 추론, GPU 시뮬레이션, 연속 배치, FlashAttention, KV 분할, TPOT

■ 서론

가) 제안배경 및 필요성

(1) LLM 서빙과 디코드 attention

Transformer[1] 구조에 기반한 대규모 언어 모델이 검색, 대화, 코드 작성 등 다양한 서비스에 활용되면서, 모델을 학습시키는 비용뿐 아니라 이미 학습된 모델로 답을 생성하는 추론(inference) 비용이 서비스 운영의 핵심 비용이 되었다. LLM 의 추론은 입력 문장 전체를 한 번에 처리하는 프리필(prefill) 단계와, 이후 토큰을 하나씩 생성하는 디코드(decode) 단계로 나뉜다. 디코드 단계는 생성할 토큰 수만큼 반복되므로 전체 생성 시간의 대부분을 차지하며, 사용자는 이를 토큰 간 지연(TPOT, time per output token)으로 체감한다.

GPU 한 장이 처리할 수 있는 연산량에 비해 요청 하나의 디코드 연산은 매우 작기 때문에, 서빙 시스템은 여러 요청을 하나의 배치로 묶어 처리한다. Orca[2]는 디코드 반복(iteration) 단위로 배치를 재구성하는 스케줄링을 제안하였고, vLLM[3]은 요청마다 길이가 다른 KV 캐시를 고정 크기 페이지 단위로 관리하는 PagedAttention 을 제안하였다. 이러한 연속 배치 방식에서는 서로 다른 시점에 도착하여 서로 다른 길이의 문맥을 가진 요청들이 같은 배치 안에서 함께 처리된다. 실제 서비스에서는 긴 문서를 요약하는 요청과 짧은 대화 요청이 섞이는 것이 일반적이므로, 한 배치 안의 KV 캐시 길이는 수십 배 이상 차이가 날 수 있다.

디코드 단계의 attention 은 새로 생성할 토큰 하나의 질의(query) 벡터가 지금까지 저장된 모든 키(key)와 값(value)을 읽어 가중합을 구하는 연산이다. 한 번 읽은 데이터에 대해 수행하는 연산이 적기 때문에 메모리 대역폭에 의해 성능이 제한되며, 병렬로 처리할 수 있는 작업의 양은 질의 길이가 아니라 배치 크기와 헤드 수에 의해 결정된다. Flash-Decoding[4]은 각 요청의 KV 캐시를 여러 조각으로 나누어 서로 다른 스레드 블록이 병렬로 처리한 뒤 결과를 합치는 KV 분할(split-KV) 방식을 제안하였고, 이 방식은 FlashAttention-2[5] 라이브러리의 디코드 경로에 포함되어 널리 사용되고 있다.

(2) 요청 길이 차이와 커널 설정 선택의 문제

KV 캐시를 몇 조각으로 나눌지(분할 수, num_splits)는 라이브러리가 배치 크기, 헤드 수, 최대 KV 길이만을 보고 휴리스틱으로 정한다. 이 휴리스틱은 배치 안의 모든 요청이 최대 길이만큼의 작업을 가진다고 가정한다. 따라서 실행되는 스레드 블록의 수가 SM 수의 일정 비율을 넘으면 GPU 가 이미 충분히 채워졌다고 판단하여 분할을 하지 않는다. RTX 4090 에서 Qwen3-4B 의 헤드 구성으로 실행하면 배치 크기가 26 이상일 때 항상 분할 없이 실행된다.

그러나 길이가 섞인 배치에서는 소수의 긴 요청을 담당한 스레드 블록에 작업이 집중되고, 짧은 요청을 담당한 블록은 일찍 종료되어 나머지 SM 이 유휴 상태가 되는 부하 불균형(load imbalance)이 발생한다. 32K 토큰 요청 1 개와 512 토큰 요청 31 개로 이루어진 배치에서는 긴 요청을 맡은 CTA 하나가 SM 하나에 돌아가는 균등한 작업 몫의 10.8 배를 처리해야 한다. 이러한 비효율은 평균 지연 시간과 같은 시스템 수준의 지표만으로는 원인을 파악하기 어려우며, 커널 수준의 측정과 GPU 실행 구조에 대한 분석이 함께 필요하다.

(3) 기존 GPU 시뮬레이터와 분석 방법의 한계

커널의 실행 구조를 분석하고 하드웨어 조건을 바꾸어 보기 위해 GPGPU-Sim[6], Accel-Sim[7]과 같은 사이클 수준 GPU 시뮬레이터가 널리 사용된다. 그러나 이들을 혼합 길이 디코드의 분석과 커널 설정 선택에 그대로 적용하기에는 다음과 같은 한계가 있었다.

첫째, 계산 비용이다. 사이클 수준 시뮬레이터는 명령어 단위로 GPU 의 동작을 재현하므로, 실제 GPU 에서 10~64 μs 동안 실행되는 attention 커널을 재생하는 데 본 작품의 환경에서 19 초에서 약 17 분이 걸렸다. 디코드 한 단계는 수십 ms 안에 끝나고 요청 길이 구성은 매 단계 바뀌므로, 사이클 수준 시뮬레이터로 후보 설정을 평가하여 그 단계의 커널을 고르는 것은 불가능하다.

둘째, 최신 GPU 에 대한 지원이다. 공개된 Accel-Sim v2.0.0 에는 실험에 사용한 RTX 4090(Ada 구조, sm_89)의 자원 구성과 명령어 대응표가 없어, 실제 GPU 에서 수집한 커널 추적을 재생할 수 없었다. GPGPU-Sim 역시 압축된 CUDA 바이너리와 디버그 줄 정보를 처리하지 못하고 일부 워프 셔플 명령의 결과가 달라, 별도의 실행 규칙 없이는 attention 커널을 올바르게 실행할 수 없었다.

셋째, 입력의 표현이다. 루프라인 모델[8]과 같은 분석적 방법이나 라이브러리의 휴리스틱은 배치 크기, 평균 또는 최대 길이처럼 배치를 대표하는 값 몇 개로 작업량을 요약한다. 이러한 요약은 총 작업량이 같더라도 요청별 길이 분포에 따라 CTA 의 종료 시점이 달라지는 꼬리 구간을 표현하지 못한다. 실제로 GPGPU-Sim 에서 총 키 수가 3,968 개와 4,096 개로 비슷한 혼합 길이 배치와 균일 길이 배치를 분할 없이 실행하면, 혼합 길이 배치가 약 8.5 배 오래 걸렸다.

이상을 정리하면 본 작품의 필요성은 다음과 같다.

- 요청별 KV 길이 분포를 입력으로 받아 CTA 단위의 실행 시간을 예측하면서도, 매 디코드 단계에서 사용할 수 있을 만큼 빠른 성능 시뮬레이터가 필요하다.
- 실험 GPU 인 RTX 4090 의 커널을 사이클 수준 시뮬레이터에서 재생하여, 부하 불균형의 기전과 자원별 민감도를 확인할 수 있어야 한다.
- 실제 서빙 조건인 가변 길이 배치에서 라이브러리의 기본 설정이 얼마나 손해를 보는지, 어떤 조건에서 손해가 커지는지 정량적으로 측정해야 한다.
- 커널 수준의 개선이 실제 사용자가 체감하는 TPOT 으로 이어지는지, 생성 결과를 유지하면서 전체 모델의 생성 과정에서 검증해야 한다.

나) 연구논문/작품의 목표

(1) 최종 목표와 세부 목표

본 작품의 최종 목표는 기존 GPU 시뮬레이터의 한계를 보강한 분석 환경을 구축하고, 이를 이용해 찾은 혼합 길이 배치의 취약 조건에서 attention 커널의 분할 수를 매 디코드 단계마다 동적으로 선택하여 실제 LLM 의 TPOT 을 줄이는 것이다. 새로운 attention 수식이나 새로운 커널을 만드는 것이 아니라, 수학적으로 같은 결과를 내는 기존 커널의 실행 설정만을 바꾼다는 점이 특징이다. 따라서 모델 가중치, 요청 순서와 배치 구성은 그대로 유지된다. 세부 목표와 최종 결과는 표 1 과 같다.

표 1. 세부 목표와 최종 결과

구분	목표	최종 결과
성능 시뮬레이터	요청별 길이를 반영하는 빠른 CTA 단위 예측	예측 오차 중앙값 2.3~12.3% 첫 선택 14.3 ms(59 배 단축)
사이클 시뮬레이터	RTX 4090 커널 재생과 기전 재현	Accel-Sim sm_89 재생 42 건 GPGPU-Sim 분할 효과 7.23 배 재현
취약 조건 관측	기본 설정의 손실과 발생 조건 정량화	6,801 개 조건, 혼합 길이 최대 12.53 배
선택 정책	테이블·모델·혼합 정책 구현	혼합 정책 최악 선택 손실 1.97~10.90%
실제 모델 검증	생성 토큰을 유지한 TPOT 개선	32K 1.81 배, 64K 2.45 배, 토큰 일치
재현과 시각화	GPU 없이 결과 재계산 및 탐색	검증 948 개 항목 통과, 대시보드

(2) 비교 범위와 평가 기준

후보는 동일한 attention 계산을 수행하는 FlashAttention-2 의 실행 변형으로, 분할 수 1, 기본 휴리스틱, 고정 분할 수 2~128 을 포함한다. 비교를 위해 FlashInfer[9]의 두 디코드 변형도 함께 측정하였다. 연속 KV 캐시와 페이지 단위 KV 캐시는 별도로 비교하며, 캐시를 비운 cold 상태와 재사용하는 warm 상태도 구분한다. 커널 선택의 품질은 선택한 실행 시간을 같은 조건의 최적 후보 시간과 비교한 선택 손실로, 실제 모델의 성능은 기본 휴리스틱 대비 TPOT 개선 배율로 평가한다.

$$\text{선택 손실} = T(\text{선택한 후보}) / \min T(\text{동일 조건의 후보}) - 1$$

$$\text{TPOT 개선 배율} = \text{TPOT(기본 휴리스틱)} / \text{TPOT(비교 정책)}$$

실제 모델에서는 같은 가중치·입력·요청 순서와 도착 시점을 사용하고 정책만 교체한다. 기본 휴리스틱과 생성 토큰이 같은 경우를 출력 동일성이 확인된 개선으로 보고, 다른 경우에는 처음 달라진 위치와 로짓 차이를 함께 분석한다. 시뮬레이터는 실측 커널 시간과 비교하여 예측 오차와 후보 순위의 일치 정도로 평가한다.

다) 연구논문/작품 전체 overview

(1) 시스템 구성

시스템은 커널 실측, 시뮬레이션 분석, 취약 조건 관측, 분할 수 선택, 실제 모델 실행, 출력·재현 검증의 여섯 단계로 구성된다. 입력은 한 배치에 포함된 요청들의 KV 길이 분포와 GPU 의 자원 정보이며, 선택 정책은 매 디코드 단계에서 이 정보를 받아 분할 수를 반환한다. 그림 1 은 각 단계의 관계를 보여 준다.



그림 1. 시스템 구성과 단계별 흐름

커널 실측 단계는 RTX 4090 에서 후보별 실행 시간과 CTA 구성을 기록한다. 시뮬레이션 분석 단계는 세 가지 도구를 역할에 따라 나누어 사용한다. 직접 구현한 CTA 이벤트 기반 성능 시뮬레이터는 요청별 길이로부터 후보의 실행 시간을 빠르게 예측하여 선택 정책에 사용되고, Accel-Sim 은 RTX 4090 구성에서 자원별 민감도를, GPGPU-Sim 은 직접 작성한 split-KV 커널로 부하 불균형의 기전을 확인한다. 선택 정책은 실측 테이블, 시뮬레이터 예측 또는 둘을 결합한 규칙으로 분할 수를 고르며, 직접 구현한 실행 엔진은 Qwen3-4B 와 Llama 계열 8B 모델을 연속 배치와 페이지 단위 KV 캐시로 실행한다.

검증 단계에서는 TPOT 과 생성 토큰 일치 여부를 반복 측정하고, 출력이 달라진 경우 분기 위치와 로짓 차이를 분석한다. 원본 기록은 재현 검증과 대시보드로 전달되므로 GPU 가 없는 환경에서도 주요 수치를 다시 계산하고 확인할 수 있다.

(2) 주요 결과와 보고서 구성

첫째, 시뮬레이션 환경을 보강하였다. CTA 이벤트 기반 성능 시뮬레이터는 실행 시간 예측 오차 중앙값 2.3~12.3%를 보였고, 공개 트레이스에서 추출한 131 개 배치 구성에서 예측과 실측 선택 손실의 순위 상관계수는 0.959 였다. C 계산 경로로 후보 9 개를 평가하는 첫 선택 비용을 14.3 ms 로 줄여, Accel-Sim 이 커널 하나를 재생하는 데 걸린 19 초보다 1,300 배 이상 짧은 시간에 판단할 수 있게 되었다. Accel-Sim 에는 RTX 4090 구성과 sm_89 재생 호환성을 추가하여 attention 커널 42 건의 재생을 완료하였다.

둘째, 혼합 길이 배치의 취약 조건을 정량화하였다. 균일 길이 배치에서 기본 휴리스틱의 선택 손실 중앙값은 0.72%로 작았지만, 혼합 길이 배치에서는 172.4%로 커졌고 최악 조건에서는 최적 설정보다 12.53 배 느린 커널이 선택되었다. 긴 요청의 길이가 8K 에서 64K 로 늘어나면 기본 설정의 실행 시간은 최선 설정 대비 중앙값 1.55 배에서 2.84 배로 늘어났다. 공개 Azure 대화 트레이스에서는 디코드 단계의 약 9~13.5%가 1.25 배 이상의 손실을 보는 반면, 짧은 요청 위주의 BurstGPT 트레이스에서는 0.3% 이하였다. 즉 긴 요청이 섞이고 배치 크기가 26 이상인 조건에서 문제가 나타난다.

셋째, 특정 시나리오에서 TPOT 을 크게 줄였다. 테이블 정책은 Qwen3-4B 의 32K 혼합 길이 조건에서 TPOT 을 1.81 배, 64K 혼합 길이 조건에서 2.45 배, 실행 도중 요청이 추가되는 조건에서 1.17 배 개선하였으며, 세 조건 모두 생성 토큰이 기본 설정과 같았다. 균일 길이에서는 기본 설정과 같은 성능을 유지하였다. 두 모델을 사용한 후속 실험 108 회에서도 성능 모델 정책이 Qwen3-4B 에서 1.28 배, Llama 계열 8B 에서 1.18 배의 개선을 보였다.

넷째, 기존 방법과 비교하였다. 커널을 바꾸지 않고 분할 수만 선택한 결과(1.81 배)는 FlashInfer 의 CUDA-core 커널로 교체한 결과(1.83 배)에 근접하였다. 또한 vLLM 에서도 혼합 길이 배치의 디코드 단계가 균일 배치보다 5.5 배 느렸고 그 차이의 97.6%가 attention 시간이어서, 본 작품이 다룬 문제가 운영용 서빙 엔진에서도 나타남을 확인하였다.

2 장에서는 LLM 서빙, attention 커널 최적화, GPU 성능 분석과 시뮬레이션에 관한 연구를 정리한다. 3 장에서는 이론적 배경, 보강한 시뮬레이션 환경, 선택 정책과 실행 엔진을 설명한다. 4 장에서는 시뮬레이션 환경의 검증, 취약 조건 관측, 실제 모델의 TPOT 개선과 기존 방법과의 비교 결과를 분석하고 한계와 향후 과제를 정리한다. 5 장은 결론과 연구 과정에서 얻은 경험을 정리한다

■ 관련연구

가) LLM 추론과 서빙 시스템

Transformer[1]는 순환 구조 없이 attention 만으로 시퀀스를 처리하는 구조로, 현재 대부분의 대규모 언어 모델의 기반이 되었다. 자기회귀 방식으로 토큰을 생성할 때에는 이전 토큰들의 키와 값을 KV 캐시에 저장해 두고 재사용한다. 본 작품의 실험에 사용한 Qwen3[10]는 0.6B 부터 235B 규모까지의 dense 및 MoE 모델로 구성된 공개 모델 계열로, 여러 질의 헤드가 하나의 키·값 헤드를 공유하는 grouped-query attention(GQA) 구조를 사용한다.

Orca[2]는 요청 전체가 끝날 때까지 기다리지 않고 디코드 반복 단위로 배치를 다시 구성하는 반복 단위 스케줄링(iteration-level scheduling)을 제안하였다. 완료된 요청은 배치에서 제외하고 새 요청을 진행 중인 실행에 추가할 수 있어, 길이가 서로 다른 생성 요청을 효율적으로 함께 처리할 수 있다. vLLM[3]은 운영체제의 가상 메모리처럼 KV 캐시를 고정 크기의 블록 단위로 관리하는 PagedAttention 을 제안하여, 가변 길이 KV 캐시로 인한 단편화와 중복을 크게 줄였다. 이들 연구가 확립한 연속 배치와 페이지 단위 KV 캐시 구조가 곧 본 작품이 다루는 혼합 길이 배치가 발생하는 환경이다.

SGLang[11]은 여러 생성 호출과 제어 흐름으로 이루어진 언어 모델 프로그램의 실행을 최적화하였다. RadixAttention 으로 공통 접두사의 KV 캐시를 재사용하고 구조화된 출력의 디코딩을 가속한다. 국내 연구에서 김수진·유헌창[12]은 질의의 의미적 유사성으로 요청을 군집화하고 대표 KV 캐시를 미리 생성하여 재사용하는 시스템을 제안하였다. 두 연구는 요청 사이에서 재사용할 수 있는 계산을 찾아 서빙 비용을 줄이는 접근이다. 본 작품은 캐시에 보존할 데이터나 배치에 포함할 요청을 바꾸지 않고 이미 구성된 배치의 attention 실행 설정을 선택하므로, 이들 기법과 함께 적용할 수 있다.

서빙 시스템의 평가에는 실제 요청의 길이와 도착 패턴이 중요하다. Splitwise[13]는 프리필과 디코드의 자원 요구가 다르다는 점에 착안하여 두 단계를 서로 다른 장비에서 실행하는 방식을 제안하였으며, 이 과정에서 Azure 의 대화·코드 서비스 요청 트레이스를 공개하였다. BurstGPT[14]는 실제 LLM 서비스의 동시성 변화와 응답 길이 분포를 담은 요청 기록을 제공한다. 본 작품은 두 공개 트레이스의 길이와 도착 시각을 재생하여 혼합 길이 배치가 실제 트래픽에서 얼마나 자주 나타나는지를 분석하였다.

나) attention 커널 최적화

FlashAttention[15]은 GPU 의 고대역폭 메모리(HBM)와 온칩 SRAM 사이의 데이터 이동을 최소화하는 타일링 기법으로, 근사 없이 정확한 attention 을 빠르고 메모리 효율적으로 계산하였다 FlashAttention-2[5]는 비행렬 연산을 줄이고 하나의 헤드도 여러 스레드 블록으로 나누어 처리하며, 워프 사이의 공유 메모리 통신을 줄이도록 작업 분할 방식을 다시 설계하여 GPU 점유율과 처리량을 높였다. 본 작품은 FlashAttention-2 라이브러리의 디코드 경로를 대상으로 한다.

Flash-Decoding[4]은 디코드 단계에서 질의 길이가 1 이기 때문에 배치가 작으면 SM 을 충분히 활용하지 못하는 문제를 지적하고, KV 캐시를 여러 조각으로 나누어 병렬로 계산한 뒤 log-sum-exp 정보로 부분 결과를 합치는 방법을 제안하였다. 조각을 많이 나누면 긴 요청의 작업을 여러 SM 에 분산할 수 있지만 부분 결과의 저장과 결합 비용도 늘어난다. 본 작품이 선택하는 대상인 분할 수가 바로 이 방법의 핵심 설정값이다.

FlashDecoding++[16]는 부분 softmax 의 동기화 비용, 작은 행렬 곱의 낮은 활용률, 고정된 실행 방식의 비효율을 비동기 softmax, flat GEMM 최적화, 입력·하드웨어별 휴리스틱 선택으로 개선하였다. 하나의 고정된 실행 구성이 모든 입력에서 최선이 되기 어렵고, 입력 형태에 따라 실행 방식을 선택해야 한다는 점은 본 작품과 공통적이다.

FlashInfer[9]는 다양한 KV 형식을 블록 희소 행렬 형식으로 통일하고, 요청별 작업량 차이를 고려하는 부하 분산 스케줄링과 JIT 컴파일 기반 attention 템플릿을 제공한다. 가변 길이 배치의 부하 불균형을 다룬다는 점에서 가장 가까운 연구이다. FlashInfer 가 새로운 커널과 내부 스케줄러로 이 문제를 해결하는 데 비해, 본 작품은 이미 널리 배포된 FlashAttention-2 커널을 그대로 두고 분할 수 인자만 외부에서 선택한다. 4 장에서는 두 접근을 같은 실행 엔진에서 비교한다.

다) GPU 성능 분석과 시뮬레이션

루프라인 모델[8]은 연산 강도를 데이터 이동량당 연산 수로 정의하고, 연산 성능과 메모리 대역폭의 상한을 함께 보아 병목을 판단한다. 디코드 attention 은 문맥 길이가 늘어날수록 읽어야 할 KV 데이터가 증가하는 메모리 병목 연산이므로 이 관점이 유용하다. 그러나 루프라인 모델은 커널 전체의 평균적인 자원 사용을 다루므로, 같은 총 데이터량이라도 요청별 길이 차이로 CTA 의 종료 시점이 달라지는 현상은 설명하지 못한다. 본 작품의 성능 시뮬레이터는 루프라인의 자원 상한을 CTA 단위의 작업 배치와 결합하여 이 한계를 보완한다.

GPGPU-Sim[6]은 CUDA 프로그램의 PTX 가상 명령어를 실행하는 상세 GPU 시뮬레이터로, 캐시, 인터커넥트, 메모리 컨트롤러, 작업 분배와 메모리 요청 병합이 성능에 미치는 영향을 분석할 수 있게 하였다. 실제 GPU 에서는 바꾸기 어려운 하드웨어 조건을 통제하여 병목의 원인을 살펴볼 수 있다는 것이 장점이다. 국내의 정종현·오윤호·구건재[17]는 GPGPU-Sim 으로 최신 GPU 아키텍처의 streaming 캐시를 분석하여, 캐시 혼잡을 줄이면 메모리 요청의 혼잡이 인터커넥트로 옮겨갈 수 있음을 보였다. 한 자원의 개선만으로 전체 병목이 해소되었다고 판단하기 어렵다는 이 결과는, 실측과 시뮬레이션을 함께 사용하는 본 작품의 분석 관점과 연결된다.

Accel-Sim[7]은 실제 GPU 에서 수집한 기계어(SASS) 명령어 추적을 이용하는 프런트엔드와 설정 가능한 성능 모델을 결합하고, 미세 벤치마크와 성능 카운터 비교를 통한 검증 방법론을 제공한다. 특히 기준 모델의 가정이 실제 하드웨어와 다르면 설계 판단이 달라질 수 있음을 보여,

시뮬레이터를 먼저 검증하고 보완해야 하는 대상으로 다루었다. 본 작품은 이 관점에 따라 공개 버전에 없는 RTX 4090 구성을 추가하고 실측과 비교하였다.

박시형 외[18]는 LLM 가속을 위한 상용 가속기의 메모리 구조, 인터커넥트와 소프트웨어 스택을 비교·정리하였다. 하드웨어마다 연산 자원과 메모리 계층이 다르므로 한 장비에서 찾은 최적 설정을 다른 장비에 그대로 적용하기 어렵다. 이는 실측만으로 답하기 어려운 하드웨어 조건의 변화를 시뮬레이션으로 분석할 필요가 있다는 근거가 된다.

라) 원안과의 관계 및 접근 방법 비교

원안은 Transformer와 선택적 상태 공간 모델 Mamba[19]를 입력에 따라 전환하는 시스템이었다. 그러나 두 모델은 구조와 가중치가 달라 생성 결과 자체가 바뀐다. 이에 프로파일링, 동적 전환, 대시보드라는 원안의 골격은 유지하면서 전환 대상을 같은 모델 안에서 수학적으로 동등한 attention 실행 설정으로 구체화하였다. 표 2는 관련 연구와 본 작품의 접근을 비교한 것이다.

표 2. 관련 연구와 본 작품의 접근 비교

연구	접근 방법	본 작품과의 관계
Orca[2]·vLLM[3]	반복 단위 스케줄링과 페이지 KV 관리	혼합 길이 배치가 발생하는 환경
SGLang[11]·김수진 외[12]	요청 간 KV 캐시 재사용	배치 구성 수준에서 함께 적용 가능
Flash-Decoding[4] FlashAttention-2[5]	KV 분할과 부분 결과 결합	선택 대상과 기본 휴리스틱
FlashDecoding++[16] FlashInfer[9]	실행 방식 선택, 커널 내부 부하 분산	비교 대상(커널 교체 방식)
GPGPU-Sim[6] Accel-Sim[7]	명령어 단위 사이클 시뮬레이션	RTX 4090 재생 보강, 기전 재현
본 작품	CTA 이벤트 시뮬레이션 기반 분할 수 선택	기존 커널 유지, TPOT·출력 검증

■ 제안 작품 소개

가) 이론적 배경

(1) 디코드 attention과 메모리 병목

요청 b의 헤드는 질의 q와 길이 L_b인 키 행렬 K, 값 행렬 V로 $o = \mathrm{softmax}(qK^{T}/\sqrt{d})V$를 계산한다. Qwen3-4B는 질의 헤드 32개가 KV 헤드 8개를 공유하는 GQA 구조이며 헤드 차원 d는 128이다. 한 단계에서 읽는 KV 양은 요청 길이의 합에 비례한다. 디코드에서는 읽는 데이터에 비해 연산량이 작아 메모리 대역폭과 병렬 작업 수가 실행 시간을 좌우한다.

(2) GPU 실행 구조와 부하 불균형

CUDA 커널은 여러 스레드 블록(CTA)의 그리드로 실행된다. 각 CTA는 한 SM에 배치되며, 동시에 상주할 수 있는 CTA 수는 레지스터와 공유 메모리 사용량의 제한을 받는다. RTX 4090은 SM 128개를 가진다. 커널은 마지막 CTA가 끝나야 종료되므로, CTA별 작업량이 다르면 짧은 작업이 먼저 끝난 뒤 일부 SM만 동작하는 꼬리 구간이 생긴다. 이 꼬리 구간의 길이는 평균 작업량이 아니라 가장 큰 CTA의 작업량이 결정한다.

(3) KV 분할의 이득과 비용

KV 분할은 각 요청·KV 헤드의 캐시를 여러 구간으로 나누어 CTA 들이 부분 attention 을 계산하고, 별도의 결합 커널이 부분 결과를 합치는 방식이다. 분할 수 S 에 따른 CTA 수는 S×B×H_kv 이다. 분할은 병렬성을 늘리지만 부분 결과를 저장·재읽기하는 트래픽과 결합 비용도 추가한다. 따라서 분할 수가 지나치게 작으면 SM 을 충분히 활용하지 못하고, 지나치게 크면 부가 비용이 증가한다.

attention 이 디코드 단계에서 차지하는 비중을 f, attention 속도 향상을 s 라 하면, 다른 연산 시간이 같을 때 단계 전체의 속도 향상은 다음과 같다. 실제 모델 결과에서는 attention 시간 변화로 예측한 단계 시간과 측정값을 비교하여 개선의 원인을 확인한다.

$$\text{단계 속도 향상} = 1 / ((1 - f) + f / s)$$

(4) 기본 휴리스틱과 혼합 길이 배치

FlashAttention 2.8.3 은 기본 CTA 수 B·H_kv·m 이 SM 당 CTA 2 개를 가정한 실행 슬롯 수의 80% 이상이면 분할하지 않는다. 그보다 작으면 1~128 범위에서 계산한 최고 활용 효율의 85% 이상을 얻는 가장 작은 분할 수를 선택한다. m 은 질의 행 블록 수이다. 이 규칙과 그리드 구성을 라이브러리 소스 코드에서 이식하여 CTA 별 작업량을 계산하였다(부록 A.1). RTX 4090 에서 KV 헤드가 8 개이면 B×8 ≥ 204.8, 즉 배치 크기 26 이상에서 항상 분할 없이 실행된다.

32K 요청 1 개와 512 토큰 요청 31 개에서는 기본 CTA 가 32×8=256 개이므로 기준 204.8 을 넘어 분할 수 1 을 고른다. 그러나 32K 요청을 맡은 8 개 CTA 에 긴 작업이 집중되고 나머지 248 개는 일찍 종료된다. 분할 수 16 에서는 긴 요청의 작업이 128 개 CTA 로 나뉘고 CTA 당 최대 키 수가 2,048 개로 줄어든다.

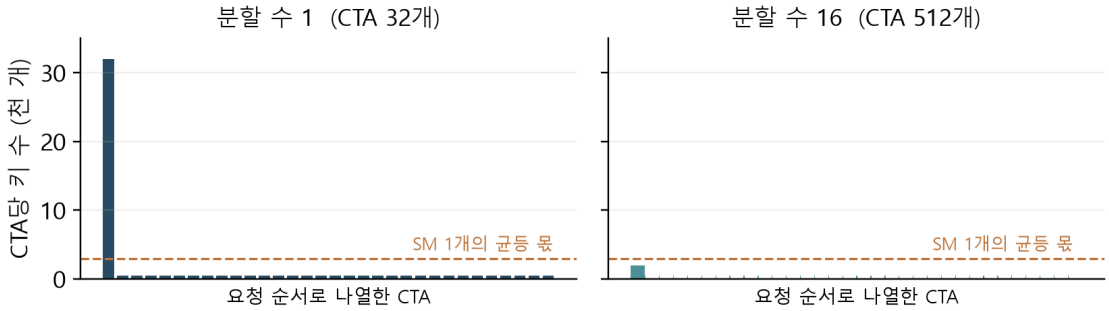


그림 2. 혼합 길이 배치(32K×1+512×31)에서 분할 수에 따른 CTA 별 키 수(KV 헤드 1 개 기준)

그림 2 는 동일한 요청 구성의 이론적 작업량을 요청 순서대로 나타낸 것이다. 분할 수 1 에서는 32K 요청의 CTA 가 SM 하나의 균등 몫보다 10.8 배 많은 키를 처리하지만, 분할 수 16 에서는 가장 큰 CTA 도 균등 몫 이하로 줄어든다. 짧은 요청의 뒤쪽 구간은 비어 있으므로 키를 처리하지 않는 CTA 도 포함된다. 즉 개선의 요인은 CTA 의 총개수를 늘리는 것이 아니라 긴 요청의 작업을 여러 SM 에 분산하는 것이다. 이 조건의 연속 KV 커널 실측 시간은 3,501 μs 에서 279 μs 로 줄었다.

나) 시스템 구성과 구현

시스템은 Python 패키지 약 12,400 줄, C 언어 시뮬레이션 코어 126 줄, 실험·분석 스크립트 약 3,000 줄, 대시보드 약 3,200 줄로 구현하였으며 단위 테스트 638 개를 작성하였다. 구성 요소는 Parquet·CSV·JSON 결과를 주고받아 GPU 측정과 CPU 분석·재현을 분리한다. 주요 모듈은 표 3 과 같다.

표 3. 주요 모듈의 구성

모듈	주요 기능	구현 위치
커널 측정기	후보별 시간·그리드·자원 기록, cold/warm 측정	kernelscope/bench
성능 시뮬레이터	CTA 구성, 이벤트 시뮬레이션, 실측 보정	kernelscope/model
사이클 시뮬레이터 연동	Accel-Sim 추적·재생·what-if GPGPU-Sim 실행과 축소 커널	kernelscope/backends experiments/gpgpusim
선택 정책	휴리스틱·고정·테이블·모델·혼합	kernelscope/serve/dispatch.py
실행 엔진	연속 배치, 페이지 KV, FA2·FlashInfer 경로	kernelscope/serve
검증·시각화	출력 분기 분석, 재현 검증, 대시보드	kernelscope/verify.py dashboard

커널 측정기는 분할 수 1, 기본 휴리스틱(num_splits=0), 고정 분할 수 2~128 의 변형을 등록하여 조건마다 후보를 비교한다. 연속 KV 와 페이지 KV 를 구분하고, CUDA 이벤트와 torch.profiler 의 CUPTI activity 로 시간·그리드·블록·레지스터·공유 메모리를 기록한다. cold 측정에서는 L2 용량의 4 배 데이터를 쓰는 커널을 먼저 실행하며, 참조 구현과 결과를 비교하고 GPU 사용률과 다른 프로세스 정보도 함께 기록한다. Nsight Compute 를 사용할 수 없는 공용 서버 환경을 고려하여 하드웨어 카운터에 의존하지 않도록 하였다.

다) 시뮬레이션 환경의 보강

1 장에서 정리한 기존 시뮬레이터와 분석 방법의 한계에 대해, 본 작품은 표 4 와 같이 시뮬레이션 환경을 보강하였다. 매 디코드 단계의 선택에는 빠른 성능 시뮬레이터를 사용하고, 사이클 수준 시뮬레이터는 RTX 4090 재생과 기전 확인에 사용하도록 역할을 나누었다.

표 4. 기존 분석 도구의 한계와 본 작품의 보강

대상	기존의 한계	본 작품의 보강
평균 길이 분석 라이브러리 휴리스틱	배치를 대표값으로 요약하여 CTA 꼬리 구간을 반영하지 못함	요청별 길이로 CTA 별 작업량을 유지하는 이벤트 시뮬레이션
사이클 수준 시뮬레이터의 비용	커널 1 개 재생에 19 초~17 분	C 계산 경로, 후보 9 개 첫 선택 14.3 ms 캐시 적중 시 수 μs
Accel-Sim v2.0.0	RTX 4090(sm_89) 구성과 명령어 대응이 없음	RTX 4090 자원 구성, sm_89 재생 호환 추적·재생·what-if 자동화
GPGPU-Sim 4.2	압축 바이너리·셔플 명령 처리 문제	실행 규칙 정립 검증된 split-KV 축소 커널 작성

(1) CTA 이벤트 기반 성능 시뮬레이터

성능 시뮬레이터의 입력은 요청별 KV 길이, 질의·KV 헤드 수, KV 저장 형식과 분할 수이다. 먼저 FlashAttention-2 의 그리드 구성 규칙에 따라 분할하지 않는 실행, 분할 실행, 페이지 KV 분할 실행을 구분하고 CTA 별 키 수의 배열을 만든다. KV 분할 구간은 최대 캐시 용량을 기준으로 정해지므로, 짧은 요청의 유효 길이를 벗어난 CTA 에는 키 처리 없이 시작·종료 비용만 부여한다. 이 구분으로 CTA 개수가 같더라도 요청 길이 분포에 따라 실제 작업량이 달라지는 상황을 표현한다.

$$\text{분할 } s \text{의 유효 키 수} = \max(0, \min(L_i - s \cdot C, C))$$

L_i: i 번째 요청의 KV 길이, C: 최대 캐시 용량을 분할 수로 나눈 구간 길이(타일 단위로 정렬)

다음으로 각 CTA 의 스레드 수, 레지스터와 공유 메모리 사용량으로 SM 당 상주 가능한 CTA 수를 계산한다. CTA 는 실제 GPU 에서 관측한 SM 배정 순서(짝수 번호 SM 다음 홀수 번호 SM)에 따라 실행 슬롯에 들어가며, 시작 비용을 처리한 뒤 키를 읽는다. 한 SM 에 CTA 두 개가 함께 상주하면 처리율 계수 γ 를 적용하고, 활성 CTA 전체의 메모리 요구량이 대역폭 상한을 넘으면 모든 CTA 의 처리율을 같은 비율로 낮춘다. 시뮬레이터는 가장 먼저 시작 단계나 작업을 끝내는 CTA 의 시점까지 시간을 진행하고, 비워진 슬롯에 다음 CTA 를 배정하는 과정을 마지막 CTA 가 끝날 때까지 반복한다(부록 A.2).

예측 커널 시간 = CTA 이벤트 실행 시간 + 실행 고정 비용 + 결합 커널 비용(S ≥ 2)

cold 상태에서는 실측 DRAM 대역폭을, warm 상태에서는 작업 집합 크기에 따른 실측 L2 적중 곡선과 DRAM·L2 대역폭 상한을 사용한다. 결합 커널 비용은 B×H_q×S 에 비례하는 항으로 모델링하였다. 이는 루프라인 모델의 자원 상한을 요청별 작업 배치와 결합한 것으로, 평균 길이 하나로는 보이지 않는 꼬리 구간을 실행 시간에 반영한다. 적용 범위는 단일 토큰 디코드, 헤드 차원 128, fp16·bf16 조건이다.

(2) 실측 기반 보정

시뮬레이터의 상수는 기계 상수와 커널 계수로 나누어 관리하였다. SM 수, DRAM 대역폭, L2 적중 곡선과 같은 기계 상수는 RTX 4090 에서 직접 측정하고, CTA 시작 비용·빈 CTA 비용·키당 처리 시간·결합 비용은 균일 길이 실측에 Nelder–Mead 방법으로 맞추었다. 기계 상수만 바꾸면 SM 수, 대역폭, L2 크기가 다른 GPU 에 대한 what-if 예측도 수행할 수 있다. 보정 결과의 정확도는 4 장 나) 에서 설명한다.

(3) 계산 경로의 개선

성능 시뮬레이터를 매 디코드 단계의 선택에 사용하려면 예측 정확도뿐 아니라 예측에 드는 시간도 중요하다. 초기 NumPy 구현은 이벤트마다 배열을 생성하고 축소하여, 새로운 길이 구성을 만날 때마다 후보 9 개를 평가하는 데 약 1 초가 걸렸다. 실제로 초기 성능 모델 정책은 혼합 길이 조건에서 attention 시간을 테이블 정책과 비슷하게 줄였으나, 첫 선택 비용 때문에 TPOT 이 45.33 ms 에 그쳤고 균일 길이와 요청 도착 조건에서는 오히려 TPOT 이 증가하였다.

이에 이벤트 계산부를 외부 의존성이 없는 C 언어로 재구현하였다. 이벤트 순서, CTA 입장 순서, 종료 판정 임계값을 그대로 유지하고, NumPy 의 합산 순서까지 재현하여 대역폭 상한의 경계에서 결과가 달라지지 않도록 하였다. 부동소수점 최적화(fast-math, 곱셈·덧셈 결합)를 끄고 컴파일하며, 컴파일러가 없거나 컴파일에 실패하면 NumPy 구현으로 자동 전환된다. NumPy 구현은 비교 기준으로 보존하였다.

또한 헤드 구성과 요청 순서를 보존한 256 토큰 페이지 수 벡터를 키로 하는 결정 캐시를 두어, 같은 길이 구성이 반복되면 시뮬레이션 없이 이전 선택을 재사용한다. 디코드가 진행되면 각 요청의 길이가 1 씩 늘어나지만 페이지 수는 256 토큰마다 바뀌므로, 대부분의 단계는 캐시에서 처리된다.

(4) 사이클 수준 시뮬레이터의 확장

Accel-Sim 은 RTX 4090 을 재생할 수 있도록 확장하였다. 기존 RTX 3070(SM86) 구성에서 출발하여 SM 128 개, L2 72 MiB, 384 비트 메모리 버스, SM 당 CTA 24 개, 블록당 공유 메모리 99 KiB, 코어 클록 2,520 MHz 의 RTX 4090 자원 구성을 만들었다. 또한 sm_89 바이너리를 기존 Ampere 명령어

대응표로 연결하는 호환 패치를 적용하여, 두 세대에 공통인 명령어로 이루어진 attention 커널을 재생할 수 있게 하였다.

추적과 재생 과정도 자동화하였다. NVBit 추적에서 워밍업과 JIT 컴파일 실행을 제외하고 대상 커널만 기록하도록 구간을 제어하였고, 추적량에 따른 예산 관리와 정상 종료 검사를 추가하였다. 한 번 수집한 추적은 기준 구성과 L2 용량, DRAM 대역폭, SM 수를 각각 2 배와 절반으로 바꾼 7 개 구성에서 재생하여 자원별 민감도를 분석한다. 이 과정으로 attention 커널 6 개 조건×7 개 구성, 총 42 건의 재생을 완료하였다.

GPGPU-Sim 4.2 는 시뮬레이터 소스를 수정하지 않고 실행 규칙으로 문제를 해결하였다. 압축하지 않은 바이너리로 컴파일하고 PTX 를 파일로 추출하여 읽도록 하였으며, 디버그 줄 정보 옵션과 결과가 달라지는 워프 셔플 명령을 피하였다. 그 위에서 온라인 softmax 와 부분 결과 결합을 포함한 split-KV attention 축소 커널을 직접 작성하였고, CPU 의 배정밀도 참조 구현 대비 최대 오차 2×10^{-8} 로 정확성을 확인하였다.

라) 분할 수 선택 정책

정책은 매 디코드 단계에서 활성 요청의 길이 목록과 헤드 구성을 받아 분할 수를 반환한다. 구현한 정책은 표 5 과 같다. 테이블·모델·혼합 정책은 앞에서 설명한 페이지 수 벡터를 캐시 키로 사용하며, 정책 계산에 드는 CPU 시간을 디코드 시간과 별도로 기록한다.

표 5. 구현한 분할 수 선택 정책

정책	선택 방법	특징
기본 휴리스틱	num_splits=0 으로 라이브러리 규칙 사용	비교 기준
고정값	항상 같은 분할 수 사용(예: 8, 16)	비교 기준
조회 테이블	가장 가까운 실측 조건의 최적 분할 수	실측 기반, 판단 비용 수 μs
성능 모델	시뮬레이터로 후보 9 개를 예측해 최소값 선택	측정하지 않은 조건에도 적용
혼합	테이블 추천이 모델 최선의 (1+δ)배 이내이면 채택	실측과 예측의 결합
통합 테이블	FA2 분할과 FlashInfer 변형을 함께 비교	plan() 비용 반영

조회 테이블 정책은 같은 헤드 구성의 실측 조건 중에서 다음 로그 특징의 거리가 가장 가까운 조건을 찾아, 그 조건의 최적 분할 수를 사용한다(부록 A.4).

$$x = (\log_2 B, \log_2 L_{max}, \log_2 L_{mean})$$

성능 모델 정책은 측정하지 않은 조건에도 적용할 수 있지만, 예측 시간이 비슷한 후보 사이에서는 순위가 뒤바뀔 수 있다. 혼합 정책은 이를 실측 정보로 보완한다. 테이블이 추천한 후보의 예측 시간이 모델이 예측한 최선 후보의 (1+δ)배 이내이면 실측 이웃의 선택을 따르고, 그렇지 않으면 모델의 최선 후보를 고른다. 본 작품에서는 δ=0.2 를 사용하였다(부록 A.3).

$$\hat{T}(\text{테이블 후보}) \le (1+\delta)\cdot\min \hat{T}(\text{후보}) \text{ 이면 테이블 후보를 선택}$$

FlashInfer 까지 후보에 넣는 통합 테이블은 층 수를 곱한 attention 시간에 FlashInfer 가 매 단계 호출하는 plan() 비용을 더해 후보를 비교한다. 한 층의 커널 시간이 아니라 실제 디코드 단계에서 회수할 수 있는 시간을 기준으로 선택하기 위함이다.

마) 실제 모델 실행 엔진과 검증 구조

(1) 실행 엔진

실행 엔진은 safetensors 형식의 bf16 가중치를 읽고 페이지 단위 KV 캐시를 관리하며, flash_attn_with_kvcache 에 정책이 선택한 분할 수를 전달한다. FlashInfer 0.6.13 의 디코드 경로도 같은 자리에 연결하여 두 라이브러리를 같은 조건에서 비교할 수 있게 하였다. 요청은 시나리오에 정의된 논리적 도착 단계 또는 공개 트레이스의 도착 시각에 따라 FIFO 순서로 배치에 추가되며, 최대 배치 크기는 64 이다. 탐욕적 디코딩으로 생성하고, 단계별 attention 시간·전체 단계 시간·정책 판단 시간과 토큰 생성 시각을 기록한다. 엔진의 수치적 정확성은 독립된 Hugging Face transformers 구현과 비교하여 확인하였다.

같은 실험 안의 모든 정책은 같은 요청 도착과 배치 순서를 보며, 각 디코드 단계의 커널 설정만 다르다. 따라서 정책 간의 TPOT 차이는 커널 설정과 그 선택 비용에서만 발생한다.

(2) 출력 검증

분할 수를 바꾸면 부분 attention 결과를 합치는 순서가 달라져 bf16 반올림 차이가 생길 수 있다. 이를 확인하기 위해 단계적인 출력 검증 과정을 구현하였다. 먼저 같은 입력과 탐욕적 디코딩에서 기본 휴리스틱과 토큰이 일치하는지 확인한다. 불일치가 있으면 요청별로 처음 달라진 위치를 찾고 기본 경로의 토큰을 입력으로 고정한 teacher forcing 실행으로 해당 위치의 로짓을 비교한다. 상위 두 후보의 로짓 차이가 bf16 표현 간격 1 개 또는 2 개 이내이면 동점(tie), 그보다 크면 명확한 분기(clear)로 분류한다.

명확한 분기는 같은 KV 상태에서 층별 커널 출력을 float32 참조와 비교하는 커널 탐침으로 다시 확인하여, 구현 오류와 반올림 차이의 증폭을 구분한다.

(3) 재현 검증과 대시보드

측정 결과는 입력 길이, 분할 수, KV 형식, 캐시 상태와 실행 환경 정보와 함께 저장한다. kernelscope verify 는 저장된 원본 기록에서 보고서의 주요 수치를 다시 계산하고, 기록과 문서의 값이 다르면 실패로 보고한다. 대시보드는 커널 지도, 성능 모델과 what-if, 서빙 결과, 후속 검증, 정책 실험의 다섯 화면과 기록 재생 및 실시간 측정을 보여 주는 시연 화면으로 구성하였다. 두 기능 모두 GPU 없이 실행된다.

■ 구현 및 결과분석

가) 실험 환경

실험 환경은 표 6 과 같다. GPU 측정은 연구실 서버의 RTX 4090 한 장에서 수행하였고, 분석·재현 검증과 GPGPU-Sim 실험은 GPU 가 없는 노트북 환경에서 수행하였다. 커널 측정은 초기 8 개 측정 집합의 6,801 개 조건에 64K 문맥 격자, 고정 분할 비교 격자, 공개 트레이스 표본 131 개 조건을 추가하였다. GPU 가 없는 환경에서 단위 테스트 [ ]개가 통과하였다.

표 6. 실험 환경

항목	내용
GPU	RTX 4090, SM 128 개, L2 72 MiB 실측 대역폭 952.6 GB/s, 텐서 코어 168.6 TFLOPS

항목	내용
소프트웨어	PyTorch 2.8.0(CUDA 12.8 빌드), CUDA 12.9, 드라이버 580.95.05 FlashAttention 2.8.3.post1, FlashInfer 0.6.13, vLLM 0.31
모델	Qwen3-4B-Instruct-2507, DeepSeek-R1-Distill-Llama-8B (bf16)
시뮬레이터	Accel-Sim v2.0.0, NVBit 1.8, GPGPU-Sim 4.2
측정 범위	균일: B=1~64, L=512~32K 혼합: B=16~64, 긴 요청 1·2·4 개, 긴 길이 8K~64K, 짧은 길이 512~2K
공개 트레이스	Azure 대화·코드 트레이스, BurstGPT

TPOT 은 요청별로 첫 토큰 이후의 연속 토큰 시각 차이를 평균한 뒤 요청과 반복에 대해 집계하였다. 실제 모델 실험은 워밍업 실행 후 정책별로 3 회 반복하였으며, 커널 실측은 10 회 워밍업 후 30 회 측정의 중앙값을 사용하였다.

나) 시뮬레이션 환경의 검증

(1) 분석 비용

표 7 는 attention 커널의 실행 시간을 얻는 데 각 도구가 소요한 시간이다. Accel-Sim 은 실제 실행 시간이 10~64 μs 인 커널을 재생하는 데 19 초에서 약 17 분이 걸렸다. 반면 성능 시뮬레이터의 C 경로는 후보 9 개를 모두 평가하는 첫 선택에 14.3 ms, 같은 길이 구성이 반복될 때는 수 μs 가 걸려 디코드 한 단계(약 34 ms) 안에서 판단할 수 있다.

표 7. 분석 도구별 평가 비용

도구	평가 대상	소요 시간
Accel-Sim	FlashDecoding B=1, L=1K 커널 2 회 실행 재생	19.1 s
Accel-Sim	FlashDecoding B=1, L=8K 커널 2 회 실행 재생	200.3 s
Accel-Sim	FlashAttention-2 B=16, L=1K 커널 2 회 실행 재생	992.6 s
GPGPU-Sim	split-KV 축소 커널 1 회(혼합 길이, 분할 1)	[ ] s
성능 시뮬레이터(NumPy)	32K 혼합 배치, 후보 9 개 첫 선택	842.6 ms
성능 시뮬레이터(C)	같은 조건	14.3 ms
결정 캐시 적중	같은 페이지 구성 재사용	0.9~5.6 μs

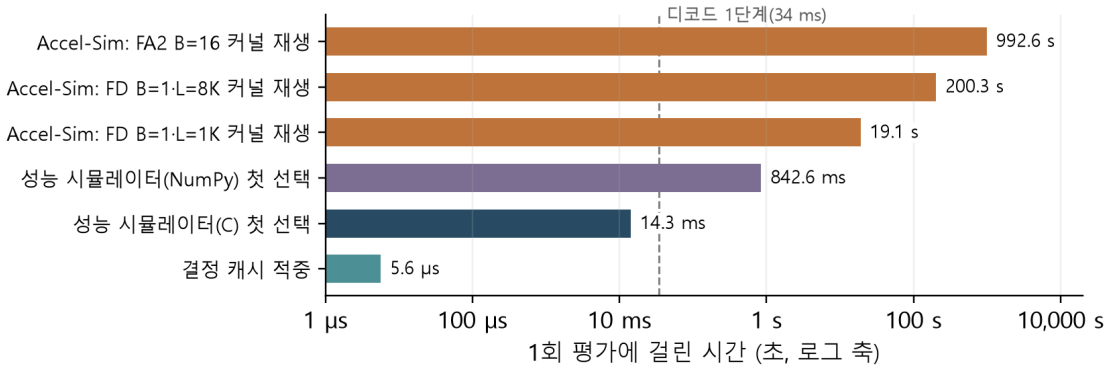


그림 3. 분석 도구별 1 회 평가 비용(로그 축)

표 8 는 C 경로의 조건별 첫 선택 비용이다. 주요 조건에서 NumPy 구현 대비 53.8~59.1 배 빨라졌다. 검증한 7 개 실제 길이 구성에서 모든 후보의 예측 시간과 선택 결과가 NumPy 구현과 정확히 같았고, 무작위 입력을 포함한 220 개 검증에서도 결과가 일치하였다. 최초 컴파일과 로드에 드는 55.7 ms 는 엔진 준비 단계에서 한 번만 발생한다. C 경로를 적용한 뒤 성능 모델과 혼합 정책은 실제 생성에서 테이블 정책과 같은 수준의 TPOT 을 얻었다(4 장 라).

표 8. 성능 시뮬레이터의 첫 선택 비용

조건	NumPy 구현	C 경로	감소 배율
균일 길이	1,068.9 ms	18.9 ms	56.5 배
혼합 길이	842.6 ms	14.3 ms	59.0 배
요청 도착·혼합	348~963 ms	6.5~17.2 ms	53.8~59.1 배
마지막 단일 요청	4.8 ms	0.5 ms	9.3 배

(2) 예측 정확도

표 9 는 보정에 사용하지 않은 검증 집합에서의 실행 시간 예측 오차이다. V1 은 균일 길이 보간, V5 는 보정에 사용하지 않은 헤드 구성, V6 는 혼합 길이 배치이다. 균일 길이로 보정한 계수가 혼합 길이 배치(V6) 1,971 개 조건에서도 오차 중앙값 11.1%를 보여, 요청별 길이 차이가 큰 조건까지 일반화되었다.

표 9. 성능 시뮬레이터의 실행 시간 예측 오차

검증 집합	조건 수	오차 중앙값	90 백분위
V1 균일 보간(cold)	594	9.0%	22.5%
V1 균일 보간(warm)	594	2.3%	13.3%
V5 미학습 헤드 구성(cold)	882	12.3%	32.1%
V6 혼합 길이(cold)	1,971	11.1%	20.2%

공개 트레이스에서 추출한 131 개 길이 구성을 RTX 4090 에서 측정하여 시뮬레이터의 선택 손실 예측도 검증하였다. 예측과 실측 선택 손실의 순위 상관계수는 0.959, 손실 오차의 중앙값은 0.048 이었다. 또한 Azure 트레이스 구간의 실제 모델 실행에서 시뮬레이터가 예측한 attention 시간 감소 비율은 1.187 배, 실측은 1.178 배로 0.8% 이내에서 일치하였다.

표 10 은 Accel-Sim 의 RTX 4090 재생 결과를 실측과 비교한 것이다. FlashAttention-2 의 B=1 조건에서는 실측의 1.04~1.06 배로 근접하였으나, Flash-Decoding 과 큰 배치에서는 1.5~3.1 배의 차이를 보였다. 같은 조건에서 성능 시뮬레이터의 예측 오차는 [ ]였다. 이에 따라 Accel-Sim 은 절대 시간보다 자원별 민감도와 기전 분석에 사용하고, 선택 정책에는 실측으로 보정한 성능 시뮬레이터를 사용하였다.

표 10. Accel-Sim 재생 결과와 실측 비교

커널·조건	실측 μs	Accel-Sim μs	비율	성능 시뮬레이터 μs
FA2 B=1, L=1K	63.6	67.5	1.06	[ ]
FA2 B=1, L=8K	494.3	515.9	1.04	[ ]
FA2 B=16, L=1K	64.3	109.7	1.71	[ ]

커널·조건	실측 μs	Accel-Sim μs	비율	성능 시뮬레이터 μs
FlashDecoding B=1, L=1K	10.0	15.5	1.55	[ ]
FlashDecoding B=1, L=8K	27.5	73.6	2.68	[ ]
FlashDecoding B=16, L=1K	38.2	116.8	3.06	[ ]

(3) GPGPU-Sim 을 이용한 기전 재현

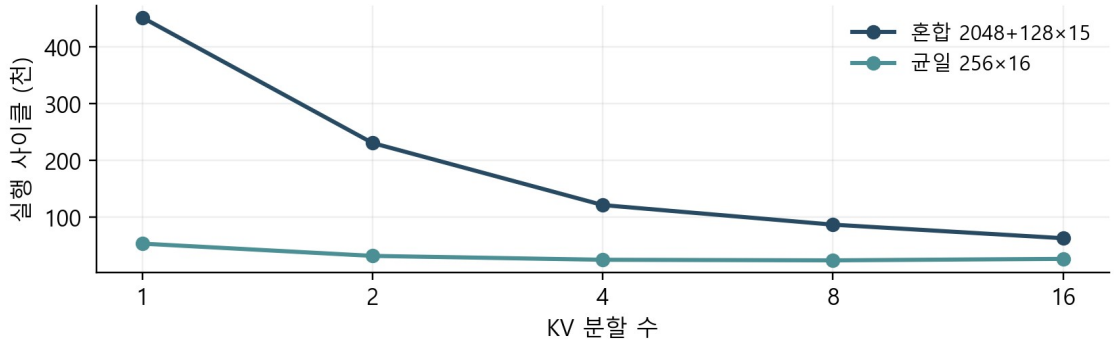


그림 4. split-KV 축소 커널의 GPGPU-Sim 실행 사이클과 분할 수

GPGPU-Sim 의 SM7_QV100 구성(SM 80 개)에서 혼합 길이 배치(2048+128×15)는 분할 1 에서 452,140 사이클, 분할 16 에서 62,566 사이클로 7.23 배 빨라졌다. 균일 길이 배치(256×16)는 분할 8 에서 2.25 배로 개선된 뒤 분할 16 에서는 결합 비용으로 다시 느려졌다. 두 배치는 총 키 수가 3,968 개와 4,096 개로 비슷하지만 분할 1 에서 혼합 길이 배치가 8.5 배 오래 걸렸다. 이는 실측에서 관찰한 경향, 즉 혼합 길이에서는 분할의 이득이 크고 균일 길이에서는 빨리 포화한다는 현상을 사이클 수준에서 재현한 것이다.

축소 커널은 fp32 와 작은 헤드 차원을 사용하므로 실제 FlashAttention-2 의 절대 시간이나 최적 분할 수를 재현하는 것은 아니지만, 하드웨어 구성을 통제한 상태에서 요청 길이 분포만으로 부하 불균형이 발생함을 실측과 독립적으로 확인하였다.

(4) Accel-Sim 의 자원 민감도와 설정 문제의 발견

RTX 4090 구성에서 자원을 하나씩 바꾸어 재생한 결과, B=16 조건에서 DRAM 대역폭을 절반으로 줄이면 실행 시간이 약 97~102% 늘어나, 디코드 attention 이 메모리 대역폭에 의해 제한된다는 분석과 일치하였다. 또한 재생 과정의 성능 카운터를 분석하여, Accel-Sim 의 L2 파티션 해시 설정이 64 개 해시 슬롯을 48 개 L2 슬라이스에 나머지 연산으로 대응시키기 때문에 0~15 번 슬라이스에 다른 슬라이스보다 약 2 배(2.003 배) 많은 읽기 요청이 집중되는 문제를 발견하였다. 이는 시뮬레이터 설정에서 비롯된 편향으로, 절대 시간 오차에 영향을 줄 수 있는 요인이다.

다) 혼합 길이 배치의 취약 조건 관측

(1) 커널 수준의 선택 손실

표 11 는 기본 휴리스틱의 선택 손실을 요약한 것으로, cold 상태의 결과이며 기본 헤드 구성 S1 은 질의 32 개·KV 8 개이다. 균일 길이에서는 휴리스틱이 최적에 가까웠지만 혼합 길이에서는 손실이 크게 증가했다. 최악의 연속 KV 조건(32K×1+512×31)은 최적 설정보다 12.53 배 느렸으며, 같은 입력의 페이지 KV 는 3.84 배였다. 실제 엔진은 페이지 KV 를 사용하므로 전체 모델의 개선은 이 경로를 기준으로 해석한다.

표 11. 기본 휴리스틱의 선택 손실 요약

측정 집합	조건 수	중앙값	최댓값	10% 초과
균일·연속 KV	49	0.7%	5.7%	0
균일·페이지 KV	49	1.2%	10.0%	0
혼합·연속 KV	162	172.4%	1152.7%	124
혼합·페이지 KV	162	43.6%	298.1%	127
혼합·페이지 KV(S2)	135	47.2%	432.3%	102

(2) 문맥 길이와 라이브러리에 따른 차이

긴 요청의 길이를 늘리면 손실이 더 커졌다. 같은 18 개 배치 구성에서 기본 휴리스틱의 실행 시간은 최선의 분할 설정 대비 중앙값 기준으로 8K 에서 1.55 배, 16K 에서 1.93 배, 32K 에서 2.56 배, 64K 에서 2.84 배였고, 64K 의 최악 조건은 5.18 배였다. 긴 문맥을 처리하는 서비스가 늘어날수록 이 문제가 커진다는 것을 의미한다.

이 문제는 FlashAttention-2 에만 국한되지 않는다. 서빙 엔진에서 GQA 구성에 따라 사용하는 FlashInfer 의 tensor-core 변형도 혼합 길이 162 개 조건 중 36 개에서 최선보다 1.25 배 이상 느렸고 최악은 2.49 배였다. 두 라이브러리 모두 균일 길이에서는 1.2 배 이내였다. 즉 요청별 길이를 보지 않는 정적인 기본 설정은 공통적으로 혼합 길이 배치에서 손해를 본다.

(3) 공개 트레이스에서의 발생 빈도

이러한 조건이 실제 트래픽에서 얼마나 자주 나타나는지 확인하기 위해, 공개 트레이스의 요청 길이와 도착 시각을 연속 배치 스케줄러(최대 배치 64, KV 캐시 10 GiB)로 재생하고 매 디코드 단계의 길이 구성을 성능 시뮬레이터로 평가하였다. 도착률을 21 가지로 바꾸어 평균 배치 크기 7.5~50 구간을 분석한 결과는 그림 5 와 같다.

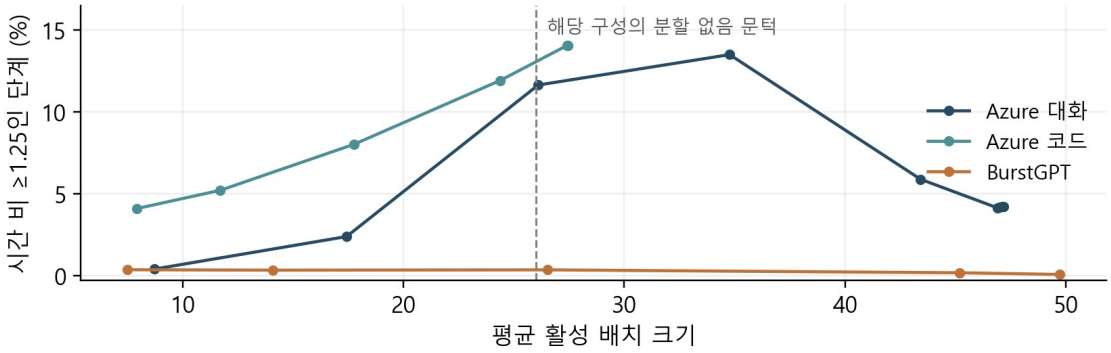


그림 5. 도착 부하에 따른 취약 단계 비율(시뮬레이터 예측)

Azure 대화 트레이스에서는 기본 휴리스틱의 손실이 1.25 배 이상인 디코드 단계가 원래 도착률에서 13.5%로 가장 많았다. 시뮬레이터가 예측한 이 비율을 검증하기 위해 손실 구간별로 층화 추출한 100 개 길이 구성을 RTX 4090 에서 측정하였으며, 실측 기반 추정치는 8.9%(95% 신뢰구간 6.3~10.9%)였다. 즉 실제 대화 트래픽에서 디코드 단계의 약 10 분의 1 이 취약 조건에 해당한다. 반면 짧은 요청 위주인 BurstGPT 는 어떤 부하에서도 0.3% 이하였다.

곡선이 봉우리 형태를 보이는 이유는 다음과 같다. 배치 크기가 26 보다 작으면 휴리스틱이 스스로 분할을 선택하므로 손실이 작고, 배치가 매우 커지면 긴 요청 하나가 전체 작업에서 차지하는

비중이 줄어든다. 따라서 문제는 긴 요청이 섞이고 배치 크기가 26 이상인 조건에서 집중적으로 나타난다. 프롬프트 길이를 4 배로 늘리고 KV 예산을 436K 토큰으로 둔 what-if 분석에서는 그 비율이 39.5%까지 증가할 것으로 예측되었다.

라) 실제 LLM 의 TPOT 개선

(1) 주요 시나리오의 TPOT 과 출력 동일성

Qwen3-4B 에서 균일 길이(512×32), 32K 혼합 길이(32K×1+512×31), 요청 도착, 64K 혼합 길이(64K×1+512×31) 시나리오를 비교하였다. 요청 도착은 16K×1+512×15 로 시작해 8·24 단계에 짧은 요청 16 개씩을 추가한다. 64K 조건은 자연어 문단을 목표 길이까지 이어 붙인 문맥을 사용하였다. 결과는 표 12 과 그림 6 과 같다.

표 12. Qwen3-4B 의 TPOT 과 출력 동일성(테이블 정책)

시나리오	기본 TPOT	테이블 TPOT	개선	토큰
균일 길이	27.60 ms	27.52 ms	1.00 배	일치
32K 혼합 길이	62.09 ms	34.25 ms	1.81 배	일치
요청 도착	59.49 ms	50.72 ms	1.17 배	일치
64K 혼합 길이	94.81 ms	38.71 ms	2.45 배	일치

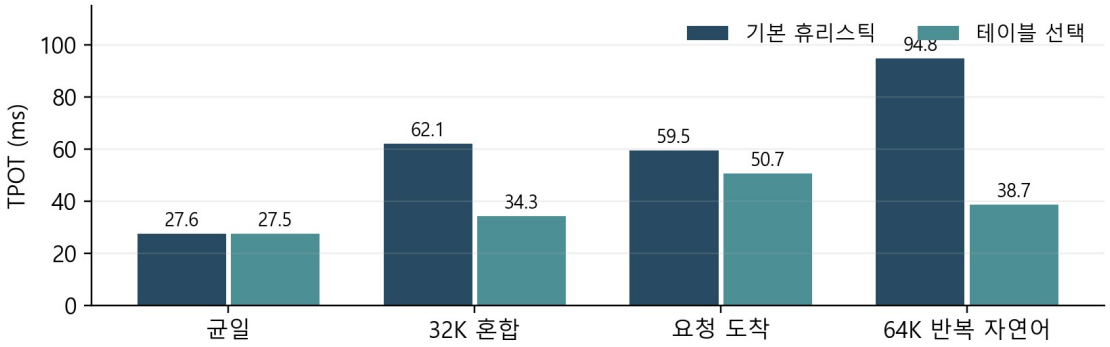


그림 6. 시나리오별 기본 휴리스틱과 테이블 정책의 TPOT

32K 혼합 길이에서 테이블 정책은 단계당 attention 시간을 36.81 ms 에서 9.25 ms 로 줄였고, 전체 단계에서 attention 이 차지하는 비중은 71.2%에서 38.3%로 낮아졌다. attention 시간 감소분만 반영하여 예측한 단계 시간(24.14 ms)과 실측(24.17 ms)의 차이는 0.1%로, TPOT 개선이 attention 커널 선택에서 비롯되었음을 확인하였다. TPOT 감소율은 44.8%이며, 중간보고서의 결과(61.06→33.80 ms, 1.81 배)를 다른 시점의 실행에서 같은 배율로 재현하였다.

64K 혼합 길이에서는 기본 설정의 attention 비중이 83.0%로 더 컸고, 테이블 정책이 attention 시간을 70.45 ms 에서 14.33 ms 로 줄여 TPOT 을 59.2% 감소시켰다. 같은 조건에서 성능 모델 정책과 혼합 정책도 각각 2.44 배의 개선을 보였으며 모두 생성 토큰이 같았다. 요청 도착 조건은 긴 요청의 비중이 작아 1.17 배였고, 균일 길이에서는 휴리스틱이 이미 최적에 가까워 정책이 기본 설정을 유지하였다.

(2) 다른 모델과 입력에서의 개선

두 모델과 직접 작성한 자연어 입력을 사용한 후속 실험 108 회(두 모델×입력 조건 3 개×시드 2 개×정책 3 개×반복 3 회)에서, 혼합 길이(12K×1+384×27) 조건의 성능 모델 정책은 Qwen3-4B 의 TPOT 을 1.282~1.290 배, Llama 계열 8B 의 TPOT 을 1.184~1.188 배 개선하였고 생성 토큰도 같았다. 모델의 크기와 구조가 달라도 같은 원리의 개선이 나타났으며, 균일 길이에서는 0.98~1.00 배로 기본 설정과 같은 수준이었다.

(3) 성능 모델과 혼합 정책

표 13 는 저장된 실측값으로 정책의 선택 품질을 평가한 결과이다. 각 조건을 제외하고 테이블을 만드는 leave-one-out 방식으로 측정하지 않은 조건에 대한 선택을 평가하였다. 성능 모델 단독으로는 예측 시간이 비슷한 후보의 순위가 뒤바뀌어 최악 선택 손실이 24~66%였으나, 혼합 정책은 cold 상태의 세 집합 모두에서 최악 손실을 기준 15% 이내(1.97~10.90%)로 줄였다.

표 13. 성능 모델과 혼합 정책의 최악 선택 손실

검증 집합	조건 수	모델 단독	혼합 정책(δ=0.2)
V1 균일 보간(cold)	594	24.3%	1.97%
V5 미학습 헤드 구성(cold)	882	66.3%	7.54%
V6 혼합 길이(cold)	1,971	24.5%	10.90%

실제 생성에서도 혼합 정책을 검증하였다. 테이블의 측정 범위에 없는 요청 도착 조건에서 혼합 정책은 일부 단계에서 시뮬레이터의 순위를 따라 테이블보다 빠른 커널을 골라, 단계당 attention 시간을 3.31 ms 에서 3.00 ms 로 줄였다. 결정 캐시를 유지하는 조건에서 TPOT 개선은 혼합 정책 1.090 배, 성능 모델 정책 1.091 배로 테이블 정책(1.086 배)보다 높았다. 판단 비용을 줄인 성능 시뮬레이터가 측정하지 않은 조건에서 실측 테이블을 보완할 수 있음을 보여 준다.

마) 기존 방법과의 비교

(1) 고정 분할 수와의 비교

분할 수를 항상 같은 값으로 고정하는 단순한 방법과도 비교하였다(표 14). 테이블 정책은 시나리오마다 분할 수를 미리 조정하지 않고도 각 시나리오에서 가장 좋은 고정 분할과 같은 수준의 TPOT 을 얻었다. 32K 혼합 길이에서 테이블 정책은 1.813 배, 고정 16 은 1.804 배였다.

표 14. 고정 분할 수와의 TPOT 개선 배율 비교

시나리오	테이블	고정 8	고정 16
32K 혼합 길이	1.813	1.644*	1.804
요청 도착	1.173	1.178	1.168
균일 길이	1.003	0.998	0.990
균일 길이 출력 분기 위치	0 곳	6 곳	6 곳

* 3 회 반복 중 원인을 확인하지 못한 느린 반복 1 회를 포함한 값이며, 이를 제외하면 1.811 배이다.

고정 분할은 길이 분포와 캐시 상태가 바뀌면 손해를 보았다. 균일 길이 생성에서 고정 8·16 은 각각 6 개 위치에서 출력이 기본 설정과 달라졌지만, 테이블 정책은 분할이 필요 없는 조건에서 기본 설정을 유지하여 분기가 없었다. 캐시를 재사용하는 warm 상태의 균일 길이 격자에서는 고정 8 이

최선 설정보다 최대 1.71 배 느렸고, 짧은 요청 위주의 BurstGPT 재생에서는 고정 8 이 기본 휴리스틱보다 최대 5.7% 느릴 것으로 예측되었다. 동적 선택은 매 단계의 길이 구성에 따라 분할 여부를 정하므로, 하나의 고정값으로는 피할 수 없는 이러한 조건별 손해에 대응할 수 있다.

(2) FlashInfer 와의 비교

표 15 는 같은 실행 엔진에서 32K 혼합 길이 조건의 TPOT 개선을 비교한 것이다. FlashInfer 의 CUDA-core 변형은 1.833 배로 가장 빨랐지만, 커널을 바꾸지 않고 FlashAttention-2 의 분할 수만 선택한 테이블 정책도 1.810 배로 그 개선의 98.8%를 얻었다. 커널 격자에서도 FlashInfer CUDA-core 변형은 최선의 FlashAttention-2 분할 설정과 같은 수준(시간 비 중앙값 0.977, 최대 1.003)이었다. tensor-core 변형은 1.257 배에 그쳤다.

표 15. FlashInfer 와의 TPOT 개선 비교(32K 혼합 길이)

방법	TPOT 개선	특징
FA2 + 테이블 정책	1.810 배	기존 커널 유지, 분할 수만 선택
FlashInfer CUDA-core	1.833 배	커널 교체, 단계당 plan() 약 0.37 ms
FlashInfer tensor-core	1.257 배	커널 교체
통합 테이블(커널 시간 기준)	1.831 배	FlashInfer 변형까지 선택
통합 테이블(plan 비용 포함)	1.810 배	FA2 분할 선택으로 수렴

FlashInfer 는 attention 커널 자체는 단계당 약 0.5 ms 빨랐지만 매 단계 plan() 호출에 약 0.37 ms 가 들어 전체 이득이 줄었다. 두 라이브러리를 함께 선택하는 통합 테이블도 plan() 비용을 반영하면 FlashAttention-2 의 분할 선택으로 수렴하였다. 즉 이 GPU 에서는 커널을 교체하지 않고 분할 수를 선택하는 것만으로 혼합 길이 배치의 개선 여지를 대부분 회수할 수 있다.

(3) vLLM 에서의 문제 확인

본 작품이 다룬 문제가 운영용 서빙 엔진에서도 나타나는지 확인하기 위해, vLLM 0.31 에서 혼합 길이와 균일 길이 배치의 디코드 단계를 torch profiler 로 분해하였다(표 16). FLASH_ATTN 백엔드에서 혼합 길이 배치의 단계 시간은 81.0 ms 로 균일 배치(14.6 ms)의 5.5 배였으며, 두 배치의 단계 시간 차이 중 97.6%가 attention 의 GPU 시간이었다. FlashInfer 백엔드에서는 혼합 길이 단계가 36.0 ms 로 줄었다.

표 16. vLLM 0.31 의 디코드 단계 분해

백엔드	배치	단계 시간	attention 시간
FLASH_ATTN	혼합 길이	81.01 ms	67.73 ms
FLASH_ATTN	균일 길이	14.63 ms	2.95 ms
FLASHINFER	혼합 길이	35.96 ms	24.03 ms
FLASHINFER	균일 길이	14.33 ms	2.96 ms

이 결과는 혼합 길이 배치의 attention 병목이 직접 구현한 실험 엔진만의 현상이 아니라, 널리 사용되는 서빙 엔진에서도 디코드 시간의 대부분을 차지함을 보여 준다.

(4) 공개 트레이스 구간의 실제 생성

공개 Azure 대화 트레이스의 요청 300 개(1,897 개 디코드 단계, 평균 배치 26.9) 구간을 실제 모델로 실행하였다. 트레이스는 요청 길이와 도착 시각을 제공하므로 입력 토큰은 시드를 고정하여 생성하였다. 테이블 정책은 TPOT 을 37.585 ms 에서 36.327 ms 로 1.035 배 줄이고 처리량을 2.8% 높였다. 이 구간의 최대 프롬프트 길이는 4,119 토큰으로 앞의 시나리오보다 짧아 개선 폭이 작았지만, 시뮬레이터가 예측한 attention 시간 감소(1.187 배)가 실측(1.178 배)과 일치하여 실제 트래픽의 길이 분포에서도 분석 환경의 예측이 유효함을 확인하였다.

바) 출력 동일성과 재현성 검증

표 12 의 네 시나리오에서 테이블 정책은 모든 반복에서 기본 설정과 같은 토큰을 생성하였다. 출력 보존을 정답 수준에서도 확인하기 위해 GSM8K[20] 200 문항을 긴 문맥 요청과 함께 혼합 길이 배치로 실행하였다. 테이블 정책과 고정 8 은 200 문항 모두 기본 설정과 같은 답을 생성하였으며 정답률도 같았다. 반면 FlashInfer CUDA-core 변형은 140 문항에서 생성 결과가 달라졌고 8 문항의 최종 답이 바뀌었다. 같은 커널 안에서 분할 수만 바꾸는 방식이 커널을 교체하는 방식보다 출력을 보존하기 쉽다는 것을 보여 준다.

출력이 달라진 조건은 3 장 마)의 검증 과정으로 분석하였다. 균일 길이에서 고정 8 이 만든 6 개의 분기 위치 중 5 개는 상위 두 로짓의 차이가 bf16 표현 간격 1 개 이내인 동점이었다. 나머지 1 개도 같은 KV 상태의 커널 탐침에서 36 개 층 모두 분할 1 과 bf16 간격 1 개 이내로 일치하고 최대 확률 토큰이 유지되어, 구현 오류가 아니라 실행 중 누적된 반올림 차이가 증폭된 것으로 확인하였다.

GPU 가 없는 노트북 환경에서 kernelscope verify 로 커널 측정, 시뮬레이터, 실제 모델 실행, 트레이스 분석의 원본 기록에서 948 개 수치를 다시 계산했으며 모두 일치하였다. 검증에는 약 [ ]초가 소요되었다.

사) 한계 및 향후 과제

본 작품의 실험은 RTX 4090 한 종류와 통제된 시나리오를 중심으로 수행되었다. 주요 한계와 향후 과제는 표 17 과 같다.

표 17. 한계와 향후 과제

항목	현재 상태	향후 과제
고정 분할 대비	혼합 길이 TPOT 은 최선의 고정 분할과 비슷	warm 상태·짧은 요청 트래픽의 실생성 비교
출력 동일성	64K 무작위 토큰 입력, 트레이스 구간에서 일부 분기	분기 원인 분류의 자동화 확대
트레이스 실생성	선택한 300 개 요청 구간, 1.035 배	대표 구간 표본, 긴 문맥 트레이스로 확대
선택 비용	새 길이 구성의 첫 결정에 수 ms	결정 캐시 사전 구축, 경량 예측
Accel-Sim 보정	실측 대비 1.04~3.06 배, L2 해시 대안 미검증	해시 설정 변경 후 재검증, Ada 타이밍 보정
운영 엔진 적용	vLLM 의 FA2 래퍼가 분할 수 0/1 만 허용	C++ 경로 수정 후 정책 이식
일반화	GPU 1 종, 헤드 구성 2 종	다른 GPU·모델의 분할 문턱 측정

특히 트레이스 구간 실행에서는 결정 캐시가 비어 있는 첫 반복에서 성능 모델과 혼합 정책이 단계당 약 6.5 ms 의 판단 비용으로 느려졌고, 세 정책 모두 일부 요청에서 출력이 기본 설정과 달라졌다. warm 상태에서는 혼합 정책의 최악 선택 손실도 41.7%로 기준을 넘었다. 선택 비용과 수치 차이를 함께 관리하는 정책이 다음 단계의 과제이다.

■ 결론 및 소감

가) 결론

본 작품은 혼합 길이 LLM 배치에서 attention 커널의 기본 분할 수 휴리스틱이 소수의 CTA 에 작업을 집중시켜 GPU 를 충분히 활용하지 못하는 문제를 다루었다. 이를 분석하기 위해 기존 GPU 시뮬레이터의 한계를 보강하였다. 요청별 길이를 CTA 단위로 반영하는 이벤트 기반 성능 시뮬레이터를 구현하여 예측 오차 중앙값 2.3~12.3%를 얻었고, C 계산 경로로 첫 선택 비용을 59 배 줄여 매 디코드 단계에서 사용할 수 있게 하였다. Accel-Sim 에는 RTX 4090 재생 기능을, GPGPU-Sim 에는 실행 규칙과 split-KV 축소 커널을 추가하여 부하 불균형의 기전을 사이클 수준에서 재현하였다.

보강한 분석 환경으로 혼합 길이 배치에서 기본 설정이 최대 12.53 배 느린 커널을 선택하고, 긴 요청이 섞이고 배치 크기가 26 이상인 조건에서 문제가 집중되며, 실제 대화 트래픽의 디코드 단계 약 10 분의 1 이 이에 해당함을 확인하였다. 이 조건에서 분할 수를 동적으로 선택하여 Qwen3-4B 의 TPOT 을 32K 혼합 길이에서 1.81 배, 64K 혼합 길이에서 2.45 배 줄였고, 생성 토큰은 기본 설정과 같았다. 커널을 교체하지 않고도 FlashInfer 로 교체한 경우에 근접한 개선을 얻었으며, 균일 길이 조건에서는 기본 설정의 성능과 출력을 그대로 유지하였다.

향후에는 고정 분할과의 비교를 warm 상태와 다양한 트래픽으로 넓히고, 결정 비용과 출력 분기를 줄이도록 정책을 보완할 계획이다. 또한 Accel-Sim 의 RTX 4090 타이밍 보정과 vLLM 등 운영 엔진으로의 이식을 진행하여 적용 범위를 확인하고자 한다.

나) 소감

가장 크게 배운 점은 문제를 정확히 정의하는 일이었다. 처음에는 Transformer 와 Mamba 를 상황에 따라 바꾸려 했지만 두 모델은 서로 다른 답을 낸다. 같은 입력의 출력을 유지하면서 실행을 개선한다는 기준으로 돌아가 전환 대상을 커널 설정으로 좁혔고, 이 기준이 이후 실험의 방향을 정했다.

시뮬레이터를 다루면서는 도구의 한계를 직접 부딪쳐 보는 경험을 하였다. Accel-Sim 에서 수십 μs 짜리 커널 하나를 재생하는 데 십수 분이 걸리는 것을 보고, 사이클 수준 시뮬레이터를 매 단계의 선택에 쓰는 것은 불가능하다는 것을 알게 되었다. 그래서 필요한 요소만 남긴 CTA 단위 시뮬레이터를 직접 만들었고, 이를 C 로 옮길 때에는 기존 구현을 기준으로 남겨 두고 모든 후보의 예측값이 같은지 확인하였다. 빠르게 만드는 것만큼 같은 계산을 한다는 것을 보이는 일이 중요하다는 것을 배웠다.

측정 과정에서는 다른 작업이 실행 중이던 공용 GPU 에서 측정한 기록에서 커널 시간이 전체 지연보다 세 배 긴 값을 발견하였다. 해당 결과를 폐기하고 재측정한 뒤 GPU 사용률과 다른 프로세스 기록을 남기도록 바꾸었다. vLLM 에서는 백엔드를 바꾸는 환경 변수가 실제로 적용되지

않아 두 실행이 같은 백엔드로 돌았던 것을 커널 이름과 로그로 뒤늦게 확인하기도 하였다. 도구가 출력한 숫자나 지정한 설정이라도 실제 실행 경로와 일관성을 확인해야 한다는 점을 배웠다.

고정 분할 수와의 비교는 결과를 다시 보게 한 계기였다. 기본 휴리스틱보다 빠르다는 것만으로는 동적 선택이 필요한 이유를 설명할 수 없었고, 단순한 고정값이 혼합 길이에서 비슷한 성능을 낸다는 결과를 받아들여야 했다. 이를 계기로 고정값이 손해를 보는 조건과 출력이 달라지는 조건을 찾아 비교하게 되었고, 성능 배율 하나보다 어떤 조건에서 왜 좋은지를 설명하는 것이 중요하다는 것을 배웠다.

출력 불일치를 분석하면서 수학적으로 같은 계산이라도 bf16 반올림 때문에 생성 토큰이 달라질 수 있다는 것을 체감하였다. 전체 토큰 차이만 세면 처음 분기한 이후의 전파가 원인을 가렸기 때문에, 첫 분기 위치와 로짓 차이를 따로 보는 도구를 만들었다. GPU 가 없는 노트북에서 검증 기능을 만들며, 결과를 얻는 것뿐 아니라 다른 사람이 같은 기록에서 수치를 확인할 수 있게 만드는 일도 연구의 일부임을 배웠다.

■ 참고문헌

[1] A. Vaswani et al., "Attention is all you need," Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[2] G.-I. Yu et al., "Orca: A distributed serving system for Transformer-based generative models," USENIX OSDI, pp. 521–538, 2022.

[3] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," ACM SOSP, pp. 611–626, 2023.

[4] T. Dao, D. Haziza, F. Massa, and G. Sizov, "Flash-Decoding for long-context inference," PyTorch Blog, 2023. https://pytorch.org/blog/flash-decoding/

[5] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," ICLR, 2024.

[6] A. Bakhoda et al., "Analyzing CUDA workloads using a detailed GPU simulator," IEEE ISPASS, pp. 163–174, 2009.

[7] M. Khairy et al., "Accel-Sim: An extensible simulation framework for validated GPU modeling," ACM/IEEE ISCA, pp. 473–486, 2020.

[8] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," Communications of the ACM, 52(4), pp. 65–76, 2009.

[9] Z. Ye et al., "FlashInfer: Efficient and customizable attention engine for LLM inference serving," MLSys, vol. 7, 2025.

[10] A. Yang et al., "Qwen3 technical report," arXiv:2505.09388, 2025.

[11] L. Zheng et al., "SGLang: Efficient execution of structured language model programs," Advances in Neural Information Processing Systems (NeurIPS), vol. 37, 2024.

[12] 김수진·유헌창, 「LLM 추론 성능 최적화를 위한 클러스터링 기반 KV 캐시 재사용 시스템」, ACK 2025 학술발표대회 논문집, 32(2), pp. 592–595, 2025.

[13] P. Patel et al., "Splitwise: Efficient generative LLM inference using phase splitting," ACM/IEEE ISCA, pp. 118–132, 2024.

[14] Y. Wang et al., "BurstGPT: A real-world workload dataset to optimize LLM serving systems," arXiv:2401.17644, 2024.

[15] T. Dao et al., "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022.

[16] K. Hong et al., "FlashDecoding++: Faster large language model inference with asynchronization, flat GEMM optimization, and heuristics," MLSys, vol. 6, 2024.

[17] 정종현·오윤호·구건재, 「최신 GPU 아키텍처의 데이터 캐시 성능 분석」, 한국소프트웨어종합학술대회 논문집, pp. 1052–1054, 2021.

[18] 박시형·이제민·김병수·전석훈, 「대규모 언어 모델 가속을 위한 상용 가속기 기술 동향」, 한국정보기술학회논문지, 23(6), pp. 75–89, 2025.

[19] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," Conference on Language Modeling (COLM), 2024.

[20] K. Cobbe et al., "Training verifiers to solve math word problems," arXiv:2110.14168, 2021.

■ 부록 A. 원천코드 주요 부분 발췌

전체 원천코드는 GitHub 저장소(https://github.com/sunjae0224/kernelscope)의 design-1-3 브랜치에 있다. 아래는 본문에서 설명한 핵심 부분을 원문 그대로 발췌한 것이다.

A.1 분할 수 휴리스틱 — kernelscope/model/geometry.py

```
def num_splits_heuristic(batch_nheads_mblocks: int, num_sms: int, num_n_blocks: int, max_splits: int
= 128) -> int:
    """Exact port of flash-attn 2.8.3 num_splits_heuristic (flash_api.cpp)."""
    if batch_nheads_mblocks >= 0.8 * num_sms:
        return 1
    max_splits = min(max_splits, num_sms, num_n_blocks)
    eff = [0.0] * (max_splits + 1)
    best = 0.0
    for s in range(1, max_splits + 1):
        if s > 1 and math.ceil(num_n_blocks / s) == math.ceil(num_n_blocks / (s - 1)):
            continue
        n_waves = batch_nheads_mblocks * s / num_sms
        eff[s] = n_waves / math.ceil(n_waves)
        best = max(best, eff[s])
    for s in range(1, max_splits + 1):
        if (s == 1 or math.ceil(num_n_blocks / s) != math.ceil(num_n_blocks / (s - 1))) and eff[s]
>= 0.85 * best:
            return s
    return 1
```

A.2 CTA 이벤트 시뮬레이션의 C 계산 경로 — kernelscope/model/_simulate_native.c

```
    while (active) {
        if (++events > max_events) { status = 1; goto cleanup; }
        double dt_fixed = INFINITY, dt_keys = INFINITY;
        int done_now = 0;
        for (size_t i = 0; i < slots; ++i) {
            rates[i] = 0.0;
            phase[i] = 0;
            if (!occupied[i]) continue;
            if (fixed[i] > EPS_US) {
                phase[i] = 1;
                if (fixed[i] < dt_fixed) dt_fixed = fixed[i];
            } else if (remaining[i] > EPS_KEYS) {
                phase[i] = 2;
                rates[i] = (residents[slot_sm[i]] >= 2 ? gamma : 1.0) / cost;
            } else {
                done_now = 1;
            }
        }
        double demand = pairwise_sum(rates, slots) * bytes_per_key;
        if (!isfinite(demand)) { status = 2; goto cleanup; }
        double scale = demand <= bandwidth ? 1.0 : bandwidth / demand;
        for (size_t i = 0; i < slots; ++i) {
            rates[i] *= scale;
            if (phase[i] == 2) {
                double duration = remaining[i] / rates[i];
                if (duration < dt_keys) dt_keys = duration;
            }
        }
        double dt = done_now ? 0.0 : fmin(dt_fixed, dt_keys);
        elapsed += dt;
        if (scale < 1.0) bw_elapsed += dt;
        if (!isfinite(elapsed) || !isfinite(bw_elapsed) || dt < 0.0) {
```

```
            status = 2;
            goto cleanup;
        }
        for (size_t i = 0; i < slots; ++i) {
            if (phase[i] == 1) fixed[i] -= dt;
            if (phase[i] == 2) remaining[i] -= rates[i] * dt;
            if (occupied[i] && fixed[i] <= EPS_US && remaining[i] <= EPS_KEYS) {
                occupied[i] = 0;
                residents[slot_sm[i]]--;
                active--;
                if (next < n) {
                    remaining[i] = keys[next++];
                    fixed[i] = remaining[i] > 0.0 ? startup : empty;
                    occupied[i] = 1;
                    residents[slot_sm[i]]++;
                    active++;
                }
            }
        }
    }
```

A.3 혼합 정책의 선택 규칙 — kernelscope/model/hybrid.py

```
def hybrid_pick(predicted: dict, table_variant, delta: float) -> str:
    best = min(predicted, key=predicted.get)
    if table_variant in predicted and predicted[table_variant] <= (1 + delta) * predicted[best]:
        return table_variant
    return best
```

A.4 조회 테이블 정책의 이웃 탐색 — kernelscope/serve/dispatch.py

```
    @staticmethod
    def _features(lens):
        return np.log2([len(lens), max(lens), float(np.mean(lens))])

    def choose(self, lens, n_heads, n_kv_heads) -> int:
        _validate(lens, n_heads, n_kv_heads)
        if self.plan_us > 0 and self.n_layers is None:
            raise ValueError("plan_us > 0 needs n_layers; Engine sets it from the model")
        key = (n_heads, n_kv_heads, tuple(math.ceil(n / PAGE) for n in lens))
        self.last_cache_hit = key in self._cache
        if self.last_cache_hit:
            value, self.attention, self.last = self._cache[key]
            return value
        eligible = self._groups.get((n_heads, n_kv_heads), [])
        if not eligible:
            raise ValueError(f"dispatch table has no measured shape with Hq={n_heads},
Hkv={n_kv_heads}")
        query = self._features(lens)
        distances = np.square(self._features_by_shape[(n_heads, n_kv_heads)] - query).sum(axis=1)
        index = int(np.argmin(distances))
        ...
```